

Gree Zone Controller — Project Summary

Custom 3-Zone HVAC Controller for Gree FXE48HP230V1R32AH

Hardware: ESP32-S3 · 3× MAX485 · SC16IS752 · SDP810-500Pa · 3-ch relay board

v0.2 — Active Development

Firmware: Arduino framework, v0.2 active development

1. The Problem

The **Gree FXE48HP230V1R32AH** is a 4-ton variable-speed ducted air handler that communicates with its controls entirely via a proprietary RS485 serial protocol (H1/H2 bus, 4800 8E1). When paired with a conventional 24V zone controller like the **Emerson EMM-3**, the unit is forced into a degraded compatibility mode that disables:

- Variable blower speed (fan runs at a fixed high speed regardless of load)
- Outdoor inverter capacity modulation (compressor runs at fixed speed)
- Energy monitoring
- All advanced heat pump features

No commercial product in the North American market solves this. This project builds a custom protocol bridge that speaks Gree RS485 natively on all buses, restoring full variable-speed operation across three independent zones.

2. Hardware Selection

ESP32-S3 DevKit

Chosen because: three hardware UARTs (critical — see §5), native USB-CDC (frees UART0 for Zone 2), built-in WiFi for Home Assistant MQTT, 3.3V GPIO, and a large ecosystem of Arduino libraries.

UART	Assignment	Baud / Format
UART1	CN port → Air handler	4800 8E1
UART2	Zone 1 thermostat RS485	4800 8E1
UART0	Zone 2 thermostat RS485	4800 8E1
SC16IS752 ch A (I2C)	Zone 3 thermostat RS485	4800 8E1

3× MAX485 RS485 Transceiver Modules

One per thermostat zone. Half-duplex operation: DE/RE pins tied together and driven by an ESP32 GPIO. Low (receive) by default; pulled HIGH briefly during the 0x2F slave response transmission, then released back to LOW.

SC16IS752 I2C Dual-UART Expander

Why needed: EspSoftwareSerial does not support 8E1 parity. The Gree packet header `0x7E` has even parity (parity bit = 0), which an 8N1 software UART misreads as a start bit on every frame — resulting in 100% packet loss. Zone 3 uses the SC16IS752 to get a true hardware UART over I2C.

SC16IS752 spec	Value
Interface	I2C (address 0x48, A0=A1=GND)
Crystal	14.7456 MHz (on-module)
Baud divisor	$14,745,600 / (4800 \times 16) = 192 \text{ exact}$
Format	8E1 (LCR = 0x1B)
TX FIFO	64 bytes (full response packet fits in one I2C burst)
Shared bus	GPIO 38/39, same I2C bus as SDP810 (no address conflict)

Sensirion SDP810-500Pa Differential Pressure Sensor

Measures supply plenum static pressure. Mounted with the high-pressure port in the plenum and the low-pressure port open to room air. Communicates via I2C (address 0x25). 6-byte frame with CRC-8 verification per reading. Used by the PID controller to modulate blower speed — maintains a constant target pressure (default 62 Pa / 0.25" WC) regardless of how many zone dampers are open. Falls back to zone-count heuristic if offline.

3-Channel Relay Board (24VAC-rated contacts)

One relay per zone damper. **Active HIGH = damper open** (spring-return motors fail-safe closed on power loss). 24VAC hot feeds through relay NO contact to the damper open terminal.

Power Supply

24VAC 40VA transformer → RAC02-05SK 5V buck converter → ESP32-S3 VIN. The same 24VAC bus powers the thermostat R/C terminals and the relay coils.

Voltage Divider (CN Port Level Shift)

The air handler CN port TX idles at 5V logic. The ESP32-S3 RX input is 3.3V max. A simple resistive divider (10kΩ / 20kΩ) reduces 5V to 3.3V on the CN_RX line. CN_TX (ESP32 → air handler) drives the 5V-logic UART directly; at 4800 baud the ESP32's 3.3V output is reliably detected as HIGH.

Gree WK-010WC1 Thermostats (×3)

Wired remote controllers switched to RS485 mode via Installer Setup (hold MODE + FAN 5 seconds). Each thermostat polls the ESP32 (acting as a fake air handler slave) every ~1 second and receives a 0x2F status response that keeps its display in sync with actual system state.

3. System Hookup Diagram

POWER

120VAC → 24VAC 40VA Transformer



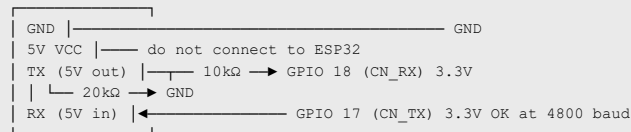
RAC02-05SK Relay board VCC Thermostat R Thermostat R
5V buck (24VAC) (Zone 1,2,3) common (C)



ESP32-S3 VIN (5V)

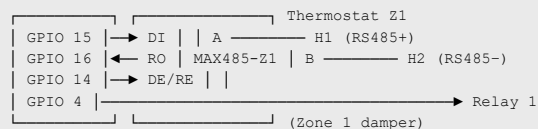
CN PORT - AIR HANDLER (UART1, 4800 8E1, 5V logic)

Gree FXE48HP230V1R32AH
GRZ19-A5 board
CN port (JST 4-pin)



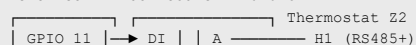
ZONE 1 - HardwareSerial UART2 (4800 8E1)

ESP32-S3 MAX485 module WK-010WC1



ZONE 2 - HardwareSerial UART0 (4800 8E1, requires USB CDC On Boot)

ESP32-S3 MAX485 module WK-010WC1



```

GPIO 12 | ← RO | MAX485-Z2 | B ——— H2 (RS485-)
GPIO 13 | → DE/RE | |
GPIO 5 | —————→ Relay 2
          (Zone 2 damper)

```

ZONE 3 – SC16IS752 I2C UART (4800 8E1)

ESP32-S3 SC16IS752 MAX485 module

```

GPIO 38 | → SDA | 14.7456MHz | TXD → DI | |
GPIO 39 | → SCL | I2C 0x48 | RXD ← RO | MAX485-Z3 |
GPIO 10 | —————→ DE/RE | |
GPIO 6 | | |
(relay) A ————— H1
(Zone 3 damper) B ————— H2
WK-010WC1
Thermostat Z3

```

SC16IS752 also wired:
VCC → 3.3V
GND → GND
A0 → GND † I2C address = 0x48
A1 → GND †

PRESSURE SENSOR – SDP810-500Pa (shared I2C bus, address 0x25)

ESP32-S3 SDP810-500Pa

```

GPIO 38 | → SDA → High-pressure port → supply plenum |
GPIO 39 | → SCL | Low-pressure port → open to room air |
3.3V | → VDD
GND | → GND
(4.7kΩ pull-ups on SDA and SCL to 3.3V)

```

RELAY BOARD → DAMPER MOTORS (spring-return, 24VAC)

For each zone:

```

Relay NO → Damper OPEN terminal (24VAC hot when relay energised)
Relay COM → 24VAC hot
Damper COM → 24VAC common (C)

```

Relay energised (GPIO HIGH) → damper opens
Relay released (GPIO LOW) → spring closes damper ← fail-safe

ESP32-S3 GPIO SUMMARY

```

GPIO 2 STATUS_LED GPIO 14 Z1_DE (MAX485 direction)
GPIO 4 Z1_RELAY GPIO 15 Z1_TX (UART2 TX)
GPIO 5 Z2_RELAY GPIO 16 Z1_RX (UART2 RX)
GPIO 6 Z3_RELAY GPIO 17 CN_TX (UART1 TX → air handler)
GPIO 8 unused* GPIO 18 CN_RX (UART1 RX ← air handler)
GPIO 9 unused* GPIO 38 I2C SDA (SDP810 + SC16IS752)
GPIO 10 Z3_DE GPIO 39 I2C SCL (SDP810 + SC16IS752)
GPIO 11 Z2_TX (UART0) GPIO 43 UART0 default TX (unused†)
GPIO 12 Z2_RX (UART0) GPIO 44 UART0 default RX (unused†)
GPIO 13 Z2_DE

* GPIO 8/9 were Z3_TX/RX (SoftwareSerial). Now free – SC16IS752 handles Zone 3.
† UART0 remapped to GPIO 11/12. Debug Serial moves to native USB-CDC.

```

4. Code Fundamental Concepts

4.1 Three-Phase Boot Sequence

The firmware starts conservatively and advances through three phases, either on a timer or manually via the `N` serial command:

```
PHASE_LISTEN (1) PHASE_POLL (2) PHASE_CONTROL (3)

CN port: RX only Send CMD_POLL (0x2C) Full zone control
Watch for 7E 7E frames Verify 0x2F response PID pressure loop
No transmission Dump bytes 16-25 Damper relays
to find room temp byte MQTT reporting
Auto-advance: 60 s Auto-advance: 30 s Runs indefinitely
```

This prevents the ESP32 from transmitting on the CN bus until it has verified that valid Gree packets are present, protecting the air handler from spurious commands during wiring commissioning.

4.2 Gree RS485 Protocol

```
Packet frame: [7E][7E][LENGTH][DATA...][CHECKSUM]
Checksum: (LENGTH + sum(DATA bytes)) % 256
Baud: 4800 8E1 (8 data, even parity, 1 stop)

Key packet types:
0x2C (44 data bytes) Control/poll - ESP32 → Air Handler
0x2F (47 data bytes) Status response - Air Handler → ESP32
Also: ESP32 → Thermostat (fake AH slave)
0x10 (16 data bytes) One-time startup handshake
```

The ESP32 plays two simultaneous RS485 roles:

1. **Master** on the CN port — sends 0x2C control packets, reads 0x2F status
2. **Slave** on each thermostat bus — receives thermostat polls, replies with 0x2F

4.3 Packet Parser (State Machine)

Each UART port has its own `Parser` instance. Bytes are fed one at a time via `parserFeed()`, which runs a four-state machine:

```
WaitH1 → WaitH2 → ReadLen → ReadData → [complete packet returned]
↑_____↓ (loop until LENGTH bytes read)
```

On completion the checksum is verified. Invalid packets are counted and discarded; the parser resets to `WaitH1` automatically.

4.4 Virtual Room Temperature

The air handler's outdoor inverter adjusts compressor speed based on the difference between the commanded setpoint and the reported room temperature. With three independent zones, a single "virtual" room temperature is

synthesised and sent to the air handler:

```
virtualRoomC = maxSetpointC - maxDeltaC

Where:
maxSetpointC = highest setpoint among all actively calling zones
maxDeltaC = largest demand delta (setpoint - roomTemp) among all zones
```

This tells the outdoor inverter to run at the capacity required by the zone with the most urgent demand, regardless of how many zones are open.

4.5 PID Pressure Control

The SDP810 measures supply plenum static pressure every 100 ms. A proportional-integral-derivative controller produces a continuous output that maps to one of three fan speeds (Low / Med / High):

```
Target: 62 Pa (0.25" WC) default - adjustable at runtime via P+/P-

PID output > 0 -> measured pressure below target -> increase fan speed
PID output < 0 -> measured pressure above target -> decrease fan speed

Hysteresis band: ±8 Pa prevents rapid speed hunting near step boundaries
```

If the SDP810 is offline, the system falls back to a zone-count heuristic:
1 zone → Low, 2 zones → Med, 3 zones → High.

4.6 Anti-Short-Cycle Protection

A four-state machine guards the compressor against rapid on/off cycling:

```
Off --[demand]--> Running --[satisfied + 3 min]--> MinOffHold
  ▲ |
  | |_____ 3 min elapsed _____|

If demand arrives during MinOffHold: dampers open, AH commanded OFF,
wait for hold to expire before starting.

If all zones satisfied during first 3 min of run: hold at Low fan until
minimum-on timer expires, then shut down.
```

4.7 Filter Monitoring

A `FilterMonitor` class tracks a slow exponential moving average ($\alpha = 0.0001$) of plenum pressure at each fan speed step. When current pressure exceeds the learned baseline by more than `FILTER_ALERT_DELTA_PA` (30 Pa default), a retained MQTT message is published to `gree_zone/filter/alert`. A loaded filter increases duct resistance, raising static pressure at a given fan speed.

4.8 MQTT / Home Assistant

All state is published every 5 seconds to a local MQTT broker (typically

Home Assistant's Mosquitto add-on). Topics are retained so the HA dashboard shows the last known value after a restart:

```
gree_zone/zone{1,2,3}/calling gree_zone/airhandler/powered
gree_zone/zone{1,2,3}/setpoint_c gree_zone/airhandler/indoor_temp_c
gree_zone/zone{1,2,3}/room_temp_c gree_zone/airhandler/mode_raw
gree_zone/zone{1,2,3}/damper gree_zone/pressure/pa
gree_zone/zone{1,2,3}/comm_ok gree_zone/pressure/target_pa
gree_zone/system/state gree_zone/filter/alert
gree_zone/system/phase
```

5. Key Challenges & Solutions

Challenge 1 — SoftwareSerial Cannot Reproduce 8E1 Parity

Problem: The Gree RS485 protocol uses 8E1 framing (8 data bits, even parity, 1 stop bit). EspSoftwareSerial only supports 8N1. The Gree packet header byte `0x7E` (0b01111110) has six 1-bits — even count — so its even parity bit is **0** (space). An 8N1 software UART interprets this parity bit as a start bit for the next byte, corrupting everything that follows. This is deterministic: **every single Gree packet header causes a framing cascade**, producing 100% packet loss on any zone using SoftwareSerial.

Solution:

- Zone 1: HardwareSerial UART2 — 8E1 native, no issue
- Zone 2: HardwareSerial UART0 remapped to Zone 2 pins — 8E1 native.

Requires "USB CDC On Boot" enabled so debug Serial moves to native USB.

- Zone 3: SC16IS752 I2C dual-UART expander — true hardware UART over I2C, full 8E1 support. 14.7456 MHz crystal divides to exactly 4800 baud (divisor = 192, zero rounding error).

Challenge 2 — Room Temperature Byte Location Unknown

Problem: The WK-010WC1 thermostat's 0x2C packet contains the room temperature at an unconfirmed byte offset. `PKT_OFF_ROOM_TEMP = 20` is a best guess from protocol reverse-engineering. Using the wrong byte would cause the virtual room temperature calculation to be incorrect, making the outdoor inverter run at the wrong capacity.

Solution: Phase 2 commissioning prints bytes 16–25 of every thermostat packet. Moving the thermostat to a warmer or cooler room and watching which byte changes reveals the correct offset. After confirmation, set

`ROOM_TEMP_BYTE_VERIFIED true` in `config.h`. Until then, the code falls

back to the air handler's own indoor sensor if the candidate byte is 0x00.

Challenge 3 — Room Temp Fallback Breaks at 16°C

Problem: `TEMP_ENCODE(16°C) = 0x00`. The original fallback condition `if (rtByte != 0x00)` would incorrectly skip a valid 16°C reading and fall back to the AH indoor sensor, causing wrong demand calculations in winter.

Solution: Added `ROOM_TEMP_BYTE_VERIFIED` flag in `config.h`. When true, the byte is always used (even if 0x00 = 16°C is a valid temperature). When false (pre-commissioning), the 0x00 guard remains active. The fallback path also adds `constrain(16, 30)` since `ah.indoorTempC` is `int8_t` and could be negative.

Challenge 4 — Half-Duplex RS485 Timing

Problem: Each thermostat bus is half-duplex. When a thermostat poll arrives, the ESP32 must switch the MAX485 DE pin HIGH, transmit the 0x2F response, then release DE back LOW — all without missing bytes from other zones. The original code used `delay(resplen × 3 ms)` after `hw->flush()`, blocking the loop for up to 265 ms per response.

Solution: `hw->flush()` already blocks until the hardware TX buffer drains (~115 ms for a 50-byte packet at 4800 8E1). The additional `delay()` was removed. For the SC16IS752, `write()` polls `LSR[TEMT]` (TX holding + shift register empty) before returning — same guarantee, no estimate required.

Challenge 5 — MQTT Reconnect Flooding

Problem: `mqtt.connect()` was called on every `loop()` iteration (~1 ms) when the broker was unreachable. `PubSubClient`'s `connect()` is blocking with a socket timeout; rapid repeated calls could lock the loop.

Solution: Added `lastMqttConnectMs` timer; reconnect attempts are rate-limited to once every `MQTT_RECONNECT_MS` (5 seconds).

Challenge 6 — Auto-Mode Zone Direction Ambiguity

Problem: Thermostats in Auto mode send mode nibble 0x80. The original code stored `ZoneMode::Auto` and later treated it as heating demand (`isHeat = dominantMode != ZoneMode::Cool`), even when the zone was actually calling for cooling (negative `deltaC`).

Solution: Auto mode is now resolved immediately at packet decode time:

deltaC > 1 → ZoneMode::Heat, deltaC < -1 → ZoneMode::Cool, otherwise
ZoneMode::Off. ZoneMode::Auto is never stored in a calling zone.

6. Phased Commissioning Timeline

```
PHASE 1 – Listen Target: ≤ 1 hour |
|
| Hardware required: Air handler only (no thermostats yet) |
|
| Goal: Confirm CN port wiring and Gree packet reception |
|
| Steps: |
| 1. Flash firmware (STARTUP_PHASE = PHASE_LISTEN) |
| 2. Open Serial Monitor @ 115200 baud |
| 3. Power air handler |
| 4. Watch for: [CN RX len=0x2F valid=Y] 7E 7E 2F ... |
|
| Pass criteria: Valid packets with correct checksum appear |
| Fail: recheck CN connector TX/RX polarity and voltage divider |
|
| 'N' or 60 s timer
▼

PHASE 2 – Poll Target: ≤ 2 hours |
|
| Hardware required: Air handler + CN port connection |
|
| Goal: Confirm bidirectional CN comms + locate room temp byte |
|
| Steps: |
| 1. Advance to Phase 2 (auto or 'N' command) |
| 2. Confirm [CN TX] poll packets appear |
| 3. Confirm [AH] status lines show plausible indoor temp |
|   e.g.: [AH] pwr:OFF mode:0x10 setpt:20°C indoor:22°C commOk:Y |
| 4. Wire Zone 1 thermostat to MAX485-Z1 |
| 5. Watch [Z1 SCAN] bytes 16-25 output |
| 6. Warm/cool the Zone 1 space – observe which byte changes |
| 7. Update PKT_OFF_ROOM_TEMP in protocol.h |
| 8. Set ROOM_TEMP_BYTE_VERIFIED true in config.h |
|
| Pass criteria: AH indoor temp plausible; room temp byte identified |
|
| 'N' or 30 s timer
▼

PHASE 3 – Control (Zone 1) Target: ≤ 1 day |
|
| Hardware required: Zone 1 thermostat + damper relay + SDP810 |
|
| Goals: Full Zone 1 operation; pressure PID baseline |
|
| Steps: |
| 1. Advance to Phase 3 |
| 2. Set thermostat to call for heat/cool |
| 3. Confirm [DAMPER] Zone 1 → OPEN |
| 4. Confirm [CTRL] lines show correct fan speed |
| 5. Confirm [AH] powered:ON |
| 6. Use P+/P- to tune target pressure for your duct system |
| 7. Verify filter monitor learns baseline after ~10 min |
| 8. Confirm MQTT topics appear in Home Assistant |
|
| Pass criteria: Zone 1 calls → AH runs → damper opens → MQTT OK |
|
| Module arrives
▼

PHASE 3 – Control (Zones 2 & 3) Target: ≤ 1 day |
|
| Hardware required: SC16IS752 installed + Zones 2 & 3 thermostats |
|
| Goals: All three zones independent; pressure PID stable |
|
| Steps: |
| 1. Install SC16IS752 (SDA=GPIO38, SCL=GPIO39, A0/A1=GND) |
```

```

| 2. Wire Zone 3 MAX485 DI/RO to SC16IS752 TXD/RXD |
| 3. Power cycle - confirm boot message: |
| [SC16IS752] chA OK - 4800 baud 8E1 (crystal=14745600Hz div=192) |
| 4. Wire Zone 2 thermostat, confirm Z2 packets decode |
| 5. Wire Zone 3 thermostat, confirm Z3 packets decode |
| 6. Test all three zones calling simultaneously |
| 7. Verify PID holds pressure as zones open/close |
| 8. Tune PID gains if needed (Kp/Ki/Kd in config.h) |
|
| Pass criteria: All 3 zones independent; pressure stable ±15 Pa |
|
|

```

```

| ONGOING - Remaining Open Items |
|
| ☐ OTA firmware updates (ArduinoOTA) - before final enclosure |
| ☐ MQTT topics for runtime PID gain adjustment (KP+/KP-) |
| ☐ Home Assistant YAML / dashboard configuration |
| ☐ Fahrenheit conversion layer (if thermostats display °F) |
| ☐ Long-term pressure baseline calibration for filter alerts |
|

```

7. Repository Branch Status

Branch	Purpose	Status
main	v0.2 baseline	stable
fix/protocol-bugs	7 correctness fixes	ready to merge
feat/zone3-sc16is752	SC16IS752 Zone 3 driver	ready — pending hardware

Merge order: fix/protocol-bugs → main **first**, then
 feat/zone3-sc16is752 → main **after** Zone 3 hardware validation.

8. File Reference

File	Purpose
firmware/GreeZoneController/GreeZoneController.ino	Main firmware — setup, loop, control logic
firmware/GreeZoneController/config.h	All tunable parameters and pin assignments
firmware/GreeZoneController/protocol.h	Gree packet structs, parser, builders, checksum
firmware/GreeZoneController/pressure.h	SDP810 driver, PID controller, filter monitor
firmware/GreeZoneController/sc16is752.h	SC16IS752 I2C UART driver (Zone 3)
firmware/GreeZoneController/secrets.h	WiFi / MQTT credentials (git-ignored)
firmware/GreeZoneController/secrets.h.template	Credential template (committed)
docs/COMMISSIONING.md	Step-by-step wiring and startup guide
docs/PROJECT_SUMMARY.md	This document

